

Python高级应用

pandas入门

1 案例背景

2 知识准备

3 任务要求

4 任务解析

不同于会计核算的相对标准化输出，可以在系统中予以固化。在财务管理过程中，常常需要处理大量的非标准化数据，而pandas是Python数据处理和分析中应用最为广泛的大型类库。pandas包含更快更简单的数据结构和处理工具，这对于财务管理过程中的分析计算以及辅助决策尤为重要。

pandas经常和其它工具一同使用，如前面学到的数值计算工具NumPy，后面将要学习的数据可视化库matplotlib。pandas是基于NumPy数组构建的，特别是基于数组的函数和不使用for循环的数据处理。虽然pandas采用了大量的NumPy编码风格，但二者最大的不同是pandas是专门为处理表格和混杂数据设计的，而NumPy更适合处理统一的数值数组数据。

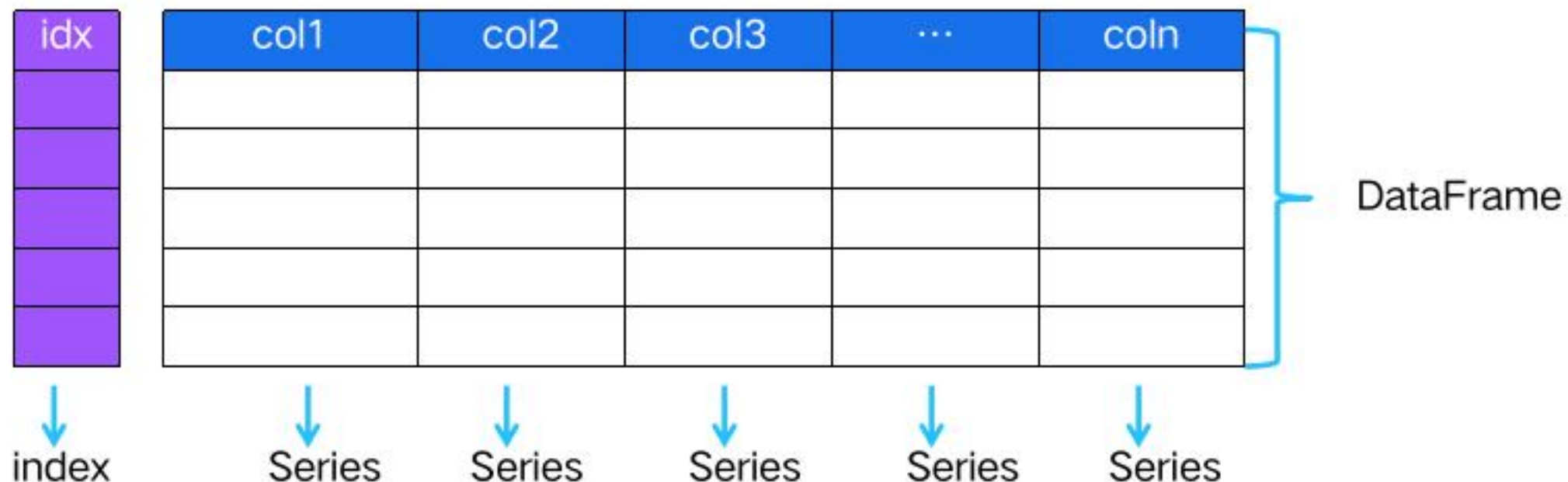
1 案例背景

2 知识准备

3 任务要求

4 任务解析

pandas最核心的数据结构包括Series和DataFrame。顾名思义，Series只有一个值“序列”，而DataFrame是可以含有多个序列的“数据框”。





Series本质是一个含有索引的一维数组，包含值序列和数据标签（索引），与Numpy的一维数组相比，Series的索引可以自定义；与Python中的字典相比，Series可以向量化运行，效率更高。

Series本质是一个含有索引的一维数组，包含值序列和数据标签（索引），可以在引入pandas后直接创建：

示例1：

```
import pandas as pd
s1 = pd.Series([2,4,6,8])
s1
```

示例2：

```
s2 = pd.Series(data = [10,20,30],index = ["一月","二月","三月"])
s2
```

DataFrame是一种表格型的数据结构，既有行索引，也有列索引，每一列可以是不同的数据类型。日常工作中DataFrame使用最为广泛，因为二维的数据本质就是一个有行有列的表格，正如我们在财务管理中处理的Excel电子表格和关系型数据库中的二维表一样。

DataFrame对象创建

有多种方式可以构建DataFrame，可以根据参数直接写入数据创建：

```
df1 = pd.DataFrame(data = [[80,400,3200],[30,500,15000],[100,500,50000]],  
                    index = ["产品A","产品B","产品C"],  
                    columns = ["数量","价格","金额"])
```

df1

DataFrame对象创建

在财务管理实务中更常用的导入excel表格或其他格式的方式获取数据，pandas通过read_excel来读取excel文件。可以在pandas的方法或属性后面加“？”来获取其使用说明：

`pd.read_excel?`

如果文件与Python文件在同一个文件路径下，可以直接输入文件名称，否则需要输入路径及文件名：

```
df2 = pd.read_excel("招商银行.xlsx","sheet1",header = 0,index_col = 0)
```

df2

基本方法和属性

如果数据表比较大，可以用.head()方法显示前几行，如果没有参数则默认显示前五

示例：`df2.head()`

info()方法可以查看对象的相关信息

示例：`df2.info()`

可以对列指定顺序排序，如果指定的列名不在数据中，则会显示为缺失值

示例：

`pd.DataFrame(df2,columns = ["净利润","营业总收入","营业利润","扣非净利润"])`

基本方法和属性

通过列标题可以检索其中的某列数据为一个序列

示例: `df2["净利润"]`

也可以将列名做为属性标记

示例: `df2.净利润`

同列表一样, 通过索引可以直接修改该列数值

示例: `df2["净利润"] = 0`

基本方法和属性

给列赋值一个Series时，序列元素的个数应与DataFrame的列长度一致

示例：

```
import numpy as np  
df2["净利润"] = np.arange(21)
```

del可以删除列

示例：`del df2["净利润"]`

drop()方法也可以用来删除行或列

示例：`df2.drop(["利润总额"],axis = 1).head()`

drop和del的区别是，del直接作用于原对象，drop删除后会返回一个新对象，如果要在原对象上删除，需要设置参数inplace为True。

基本方法和属性

使用前面学过的numpy方法也可以对DataFrame进行转置

示例: `df2.T`

通过特殊属性loc查找数据, 可以使用行标签和列标签定义数据坐标并获取数据

示例:

`df2.loc[2021,["营业总收入","营业总成本"]]`

iloc则使用整数标签, 即行或列的序列来定义获取数据

示例: `df2.iloc[2,[0,1]]`

基本方法和属性

sort_index可以对数据按行标签进行排序

示例: `df2.sort_index()`

也可以按一个或多个列的值进行排序

示例:

`df2.sort_values(by = ["营业总收入","利润总额"])`

数据运算与统计

两个DataFrame可以进行相加，相加后的数据是两个DataFrame的并集，如果存在未交叠的标签位置，则返回缺失值

示例：

```
df3 = df2.loc[:2019,["营业总收入","营业总成本","营业利润"]]
```

```
df4 = df2.loc[:2018]
```

```
df3 + df4
```

相减的运算也是类似：

示例：

```
df3 - df4
```

数据运算与统计

在df3上使用add方法，将df4做为参数传入，，如果有缺失值可以通过fill_value参数进行填充，将会获取df4中的数据对df3进行补全

示例：`df3.add(df4,fill_value = 0)`

Numpy的通用函数对pandas对象也有效

示例：

`np.sqrt(df3)`

数据运算与统计

当然，pandas本身也配置了常用的数学、统计学方法的集合，例如sum()方法可以对各列数据进行求和

示例：`df3.sum()`

如果求解一行上各列数据的和，则传入axis参数值为1或“columns”

示例：`df3.sum(axis = 1)`

cumsum()方法可以对数据进行累积求和

示例：

```
df5 = df2.sort_index()
```

```
df5.cumsum()
```

cumsum()方法可以对数据进行累积求和

示例：`df5.describe()`

函数	说明
add,radd	加法
sub,rsub	减法
div,rdiv	除法
floordiv,rfloordiv	整除
mul,rmul	乘法
pow,rfpow	幂次方
count	非NA值的个数
describe	计算Series或DataFrame各列的汇总统计集合
min,max	计算最小值，最大值
argmin,argmax	分别计算最小值、最大值所以在索引位置
idxmin,idxmax	分别计算最小值、最大值所在的索引标签
quantile	计算样本从0到1的分位数
sum	求和

函数	说明
mean	均值
median	中位数（50%分位数）
mad	平均值的平均绝对偏差
prod	所有值的积
var	值的样本方差
std	值的样本标准差
skew	样本偏差值
kurt	样本峰度值
cumsum	累计值
cummin,cummax	累计值的最小值、最大值
cumprod	值的累计积
diff	计算第一个算术差值（常用于时间序列）
pct_change	计算百分比

练习2.2-1：导入附件“工资表.xlsx”为DataFrame对象，统计该公司工资总额、平均工资和中位数工资。



处理缺失值

在pandas中，缺失值显示为NA(not available)，isnull方法可以判断对象中的数据是否为缺失值

示例：`df6.isnull()`

dropna方法会根据每个标签的值是否缺失来筛选轴标签，默认情况下dropna会删除包含缺失值的行

示例：`df6.dropna()`

参数how = “all” 时，将删除所有值均为NA的行

示例：`df7 = df6.dropna(how = "all")`

如果想要删除NA值所在的列，传入参数axis = 1

示例：`df7.dropna(axis = 1)`

处理缺失值

fillna方法可以用来补全缺失值，传入拟补全的值即可

示例：`df6.fillna(0)`

fillna返回的是一个新的对象，如果要修改原对象，需要传入参数`inplace = True`

示例：`df6.fillna(0,inplace = True)`

删除重复值

drop_duplicates()方法用于删除重复数据

示例：

```
df8 = pd.concat([df6,df7])
```

```
df8.drop_duplicates()
```

数据替换

replace()方法可以对数据进行替换

示例：

```
df8.replace(0,1000)
```

练习2.2-2：导入附件“销售订单.xlsx”为DataFrame对象，删除订单编号重复的数据行，将物料名称中的“柴油仓储叉车”替换为“电动仓储叉车”，对表格中的“价格”列的缺失值以该行平均值进行插补。



包含在pandas对象的数据可以通过多种方式联合在一起。通过merge可以根据一个或多个标签将行进行连接。

merge

可以通过pd.merge?查看merge的相关参数及用法

示例: `pd.merge?`

如果没有指定列标签, merge会自动将重叠的列名作为连接的标签

示例:

```
df9 = df2.loc[:2018]
```

```
df10 = df2.loc[2019:2014]
```

```
pd.merge(df9,df10)
```

默认情况下，merge做的是“内连接”，即参数how = “inner”，获得的结果是两张表的交集。参数how还可以分别设置为“left”、“right”、“outer”，其代表的意义如下所示：

选项	行为
“inner”	对两张表均有的行标签的交集进行合并
“left”	按左表的行标签进行合并
“right”	按右表的行标签进行合并
“outer”	对两张表所有的行标签的并集进行合并

concat

concat可以根据索引将不同对象的值进行连接。

示例：

```
df11 = df2.loc[:2018,["营业总收入","营业总成本"]]  
df12 = df2.loc[:2018,["营业总收入","营业利润","利润总额"]]  
pd.concat([df11,df12])
```

默认情况下concat的连接方式是外连接（join = “outer”），设置内连接时，选择join = “inner”

示例：

```
pd.concat([df11,df12],join = "inner")
```

文件导出

DataFrame文件操作完成后通过to_excel方法将文件写入excel文件，可以在Python文件夹中找到导出的文件：

示例：

```
df12.to_excel("写入文件.xlsx")
```

练习2.2-3：基于练习2.2-2处理完成的DataFrame对象，针对每个订单编号计算一列金额,与原表中的订单编号组成一个新的DataFrame，并与原对象按订单编号连接，完成后导出为新的excel表格。



1 案例背景

2 知识准备

3 任务要求

4 任务解析

按以下步骤对金三江各期的财务报表进行杜邦分析：

- (1)在Python环境读取提供的金三江2020-2022年度资产负债表和利润表摘要数据为DataFrame对象；
- (2)对报表数据进行转置；
- (3)按报表时间合并资产负债表和利润表项目；
- (4)计算各期间的销售净利率、总资产周转率、权益乘数和净资产报酬
- (5)导出计算指标为excel文件。

注：总资产周转率及净资产报酬率的总资产均取期末数。



1 案例背景

2 知识准备

3 任务要求

4 任务解析

根据任务要求，设计Python代码编写的流程。



- ✓ 在Python环境读取提供的金三江2020-2022年度资产负债表和利润表摘要数据为DataFrame对象；
- ✓ 观察报表字段，对报表数据进行转置；
- ✓ 按报表时间合并资产负债表和利润表项目；
- ✓ 计算各期间的销售净利率、总资产周转率、权益乘数和净资产报酬率；
- ✓ 导出计算指标为excel文件。



```
import pandas as pd
import numpy as np

balance = pd.read_excel("资产负债表.xlsx","Sheet1",header = 0,index_col = 0).T
profit = pd.read_excel("利润表.xlsx","Sheet1",header = 0,index_col = 0).T

finance = pd.concat([balance,profit],axis = 1)

finance["销售净利率"] = finance["净利润"]/finance["营业收入"]
finance["总资产周转率"] = finance["营业收入"]/finance["资产总计"]
finance["权益乘数"] = finance["资产总计"]/finance["股东权益合计"]
finance["净资产报酬率"] = finance["净利润"]/finance["资产总计"]

finance.drop(["资产总计","股东权益合计","营业收入","净利润"],axis = 1,inplace = True)
finance.to_excel("杜邦分析.xlsx")
```



通过Python批量汇总附表中的全国七大片区的销量情况，在汇总表格中添加一行计算全国各月份销量，并将汇总表格导出excel。

（提示：利用本章任务一中的os库遍历目录和拼接目录可以让程序更简洁）



感謝
ขอบคุณ
terima kasih
ありがとう
Thanks
谢谢